Neural Networks Fundamentals

Prof. Surya Prakash
Department of Computer Science & Engineering
Indian Institute of Technology Indore, Indore-453552, INDIA
E-mail: surya@iiti.ac.in

Content

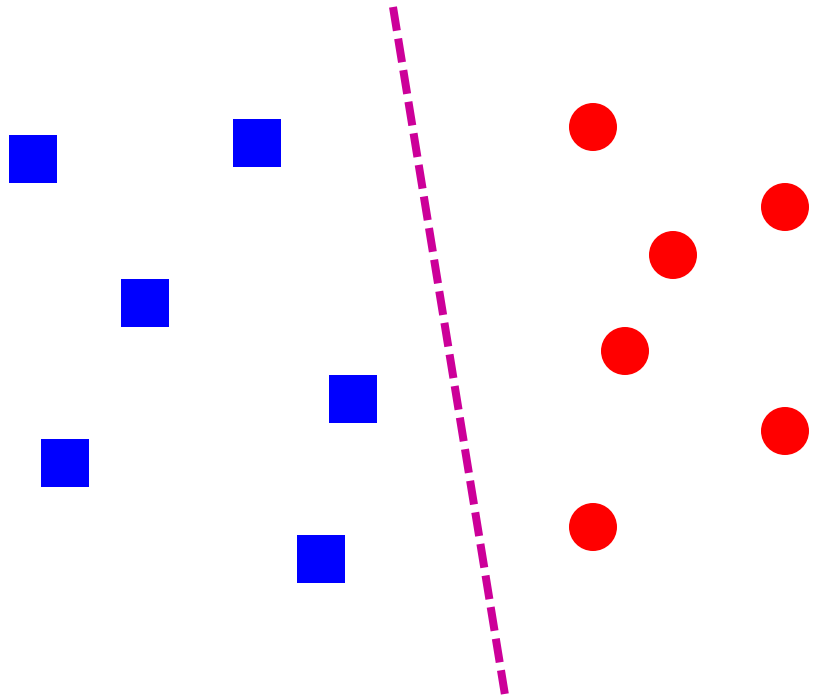
- Motivation
- Brain vs. Computers
- The Perceptron
- Multilayer networks
- Some Applications

Motivation for Machine Learning

Supervised Learning (Classification problems)

- **Motivation:**
  - Many real-world problems require **predicting an output from labeled data**.
  - Writing explicit rules for such tasks is difficult.
- **Tasks:**
  - Classification (spam / not spam)
  - Regression (house price prediction)
- **Why ML?**
  - The system learns the input–output mapping automatically from examples.

Classifiers: Linear



- Find a *linear function* to separate the classes: $f(\mathbf{x}) = \text{sgn}(\mathbf{w} \cdot \mathbf{x} + b)$
- In 2D, $\mathbf{w}=[w_1,w_2]$, $\mathbf{x}=[x_1,x_2]$

$$f(\mathbf{x}) = \text{sgn}(w_1x_1+w_2x_2+b)$$

Slide credit: L. Lazebnik

Motivation for Machine Learning

Unsupervised Learning (Clustering problems)

- **Motivation:**
  - In many cases, **labeled data is not available**.
  - We still want to discover structure or patterns in data.
- **Tasks:**
  - Clustering (customer segmentation)
  - Dimensionality reduction (feature extraction)
- **Why ML?**
  - The system uncovers hidden patterns without human supervision.

Motivation for Machine Learning

Semi-Supervised Learning

- **Motivation:**
  - Labeling data is **expensive and time-consuming**.
  - Large amounts of unlabeled data are available.
- **Tasks:**
  - Text classification
  - Image recognition with few labeled samples
- **Why ML?**
  - Combines a small labeled dataset with large unlabeled data to improve learning.

Motivation for Machine Learning

Reinforcement Learning

- **Motivation:**
  - Some problems involve **sequential decision-making**, not fixed labels.
  - Feedback comes as **reward or penalty**, not correct answers.
- **Tasks:**
  - Game playing (Chess, Go)
  - Robotics and control systems
- **Why ML?**
  - The system learns optimal behavior through trial and error.

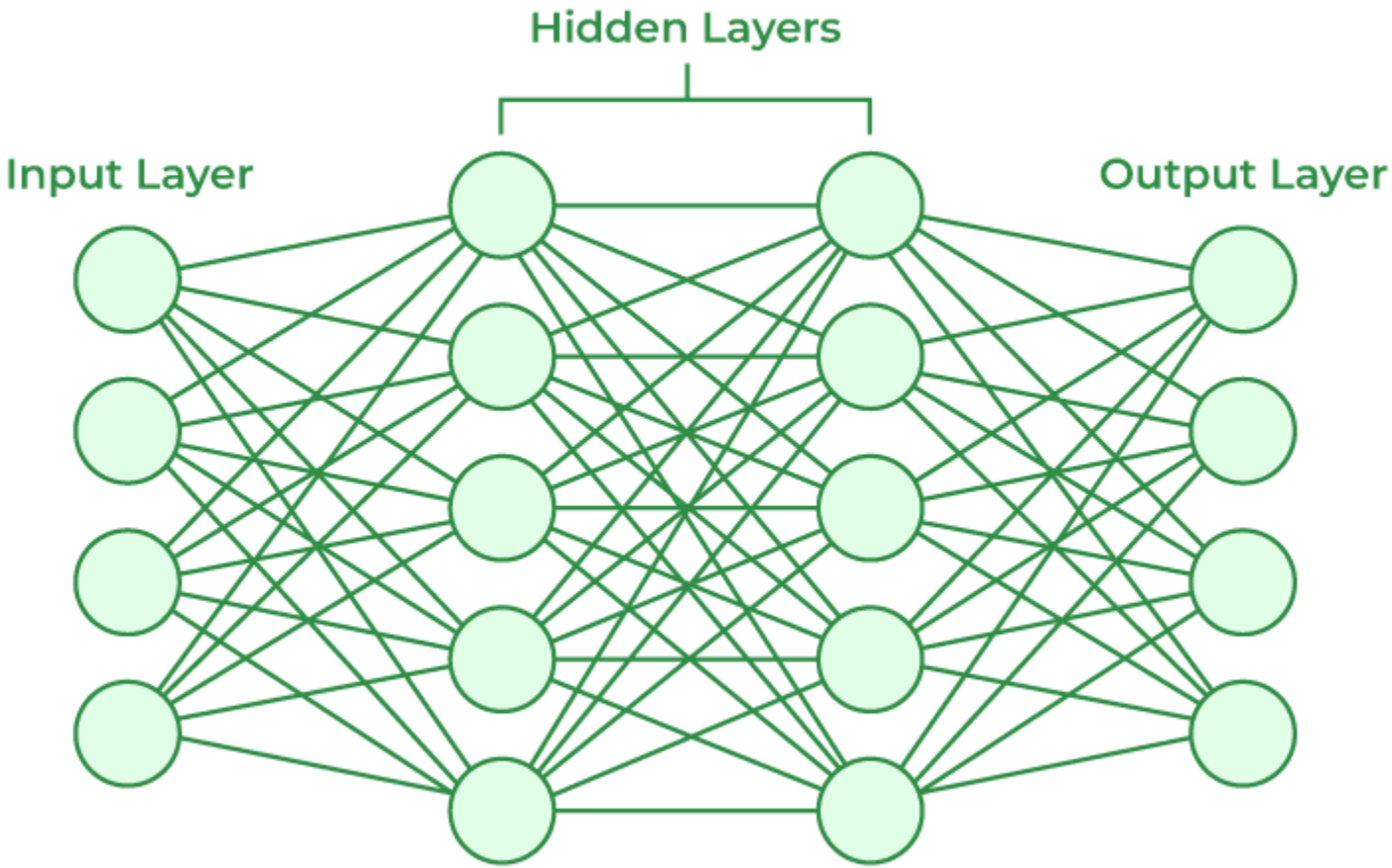
Machine Learning Techniques

Supervised	Unsupervised	Semi-Supervised	Reinforcement
Decision Tree	k-Means	Self-Training	Q-Learning
SVM	PCA	Label Propagation	SARSA
Neural Network (MLP)	Autoencoder (Neural Network)	Semi-Supervised Neural Network	Deep Q-Network

Neural Networks

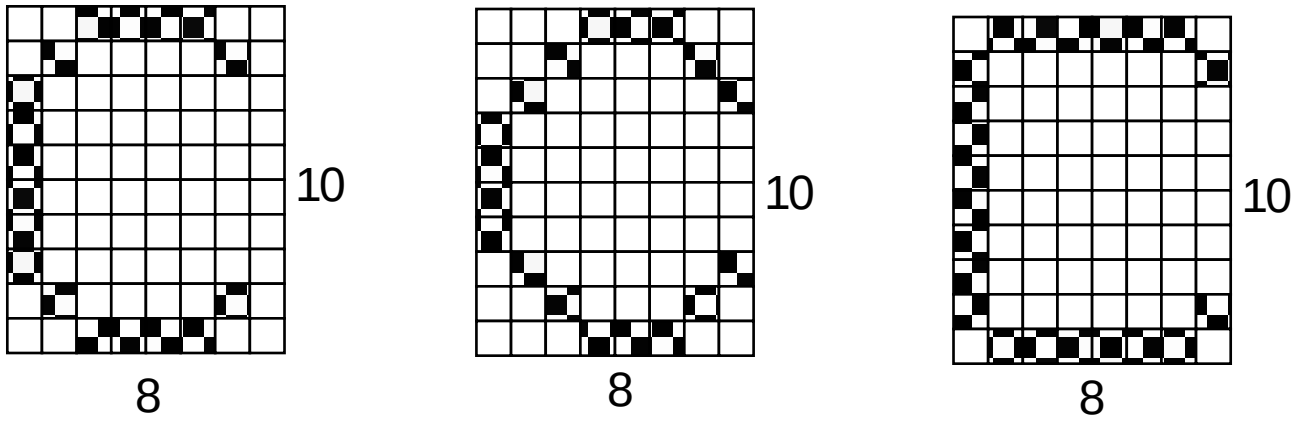
- Neural Networks are computational models inspired by the human brain.
  - They consist of interconnected neurons (nodes) arranged in layers.
  - Each connection has an associated weight, which is adjusted during learning.
  - Neural networks learn from data by minimizing error using learning algorithms such as backpropagation.

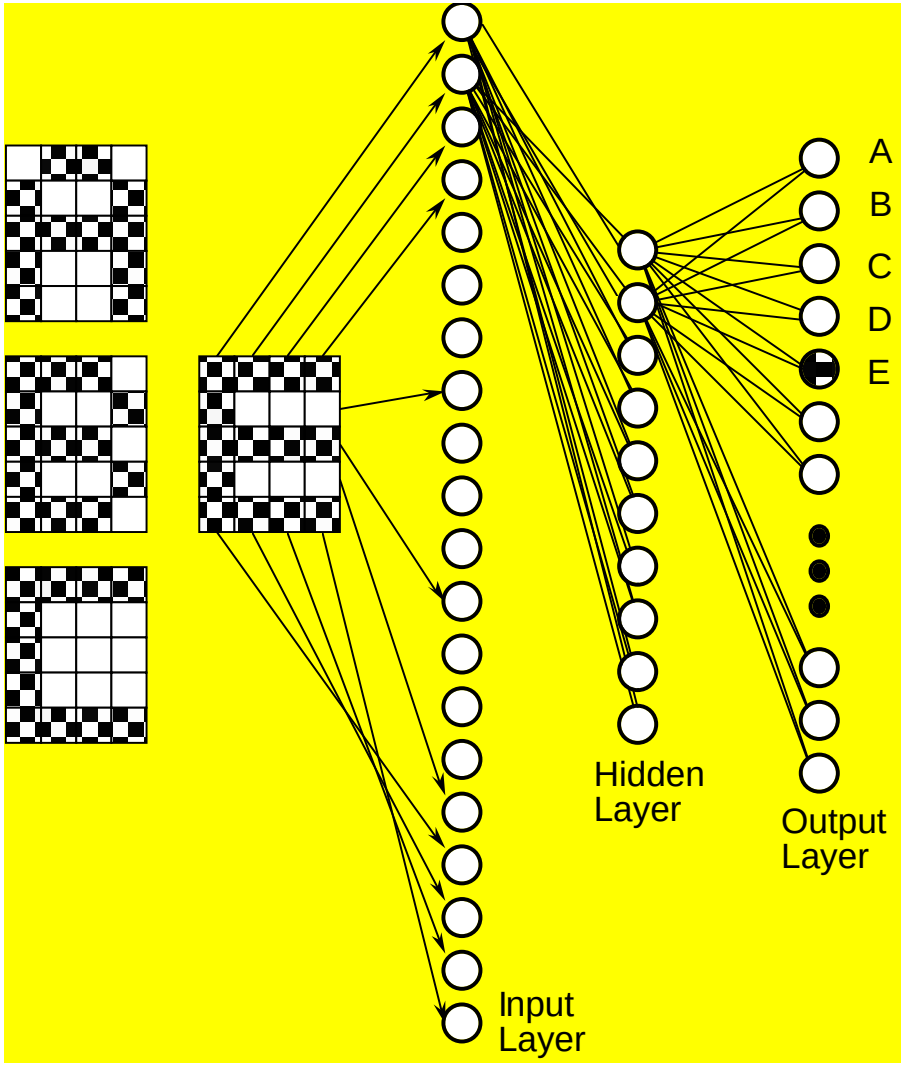
Neural Networks



Neural Network for OCR

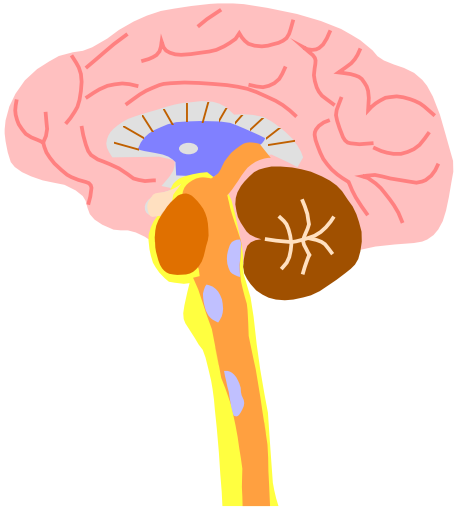
- **OCR for 8x10 Characters**
  - Feedforward network
  - Trained using Backpropagation

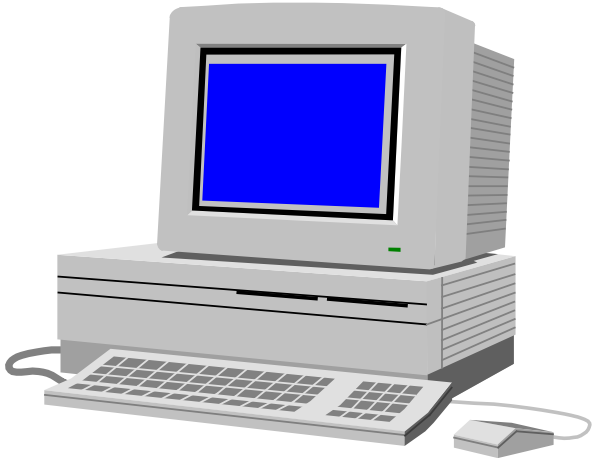




Brain and Machine

- The Brain
  - Pattern Recognition
  - Association
  - Complexity
  - Noise Tolerance

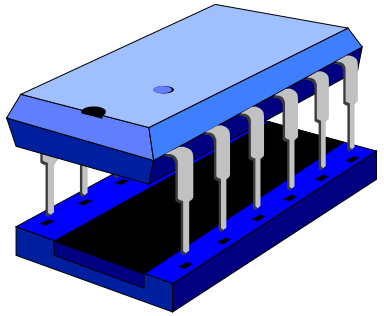




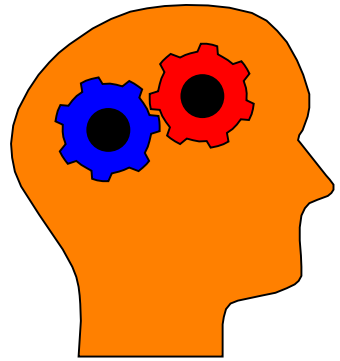
- The Machine
  - Calculation
  - Precision
  - Logic

The Contrast in Architecture

- The Von Neumann architecture uses a single processing unit;
  - Tens of millions of operations per second
  - Absolute arithmetic precision

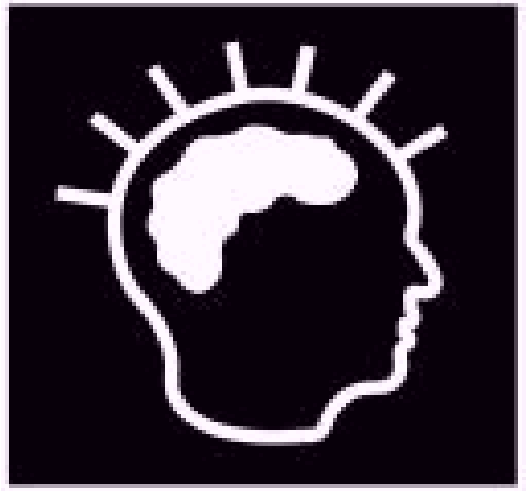


- The brain uses many slow unreliable processors acting in parallel



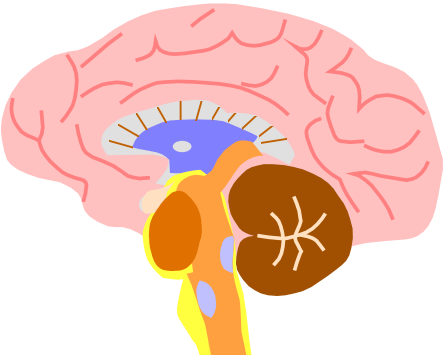
Features of the Brain

- Ten billion (10^{10}) neurons
- On average, several thousand connections
- Hundreds of operations per second
- Die off frequently (never replaced)
- Compensates for problems by massive parallelism

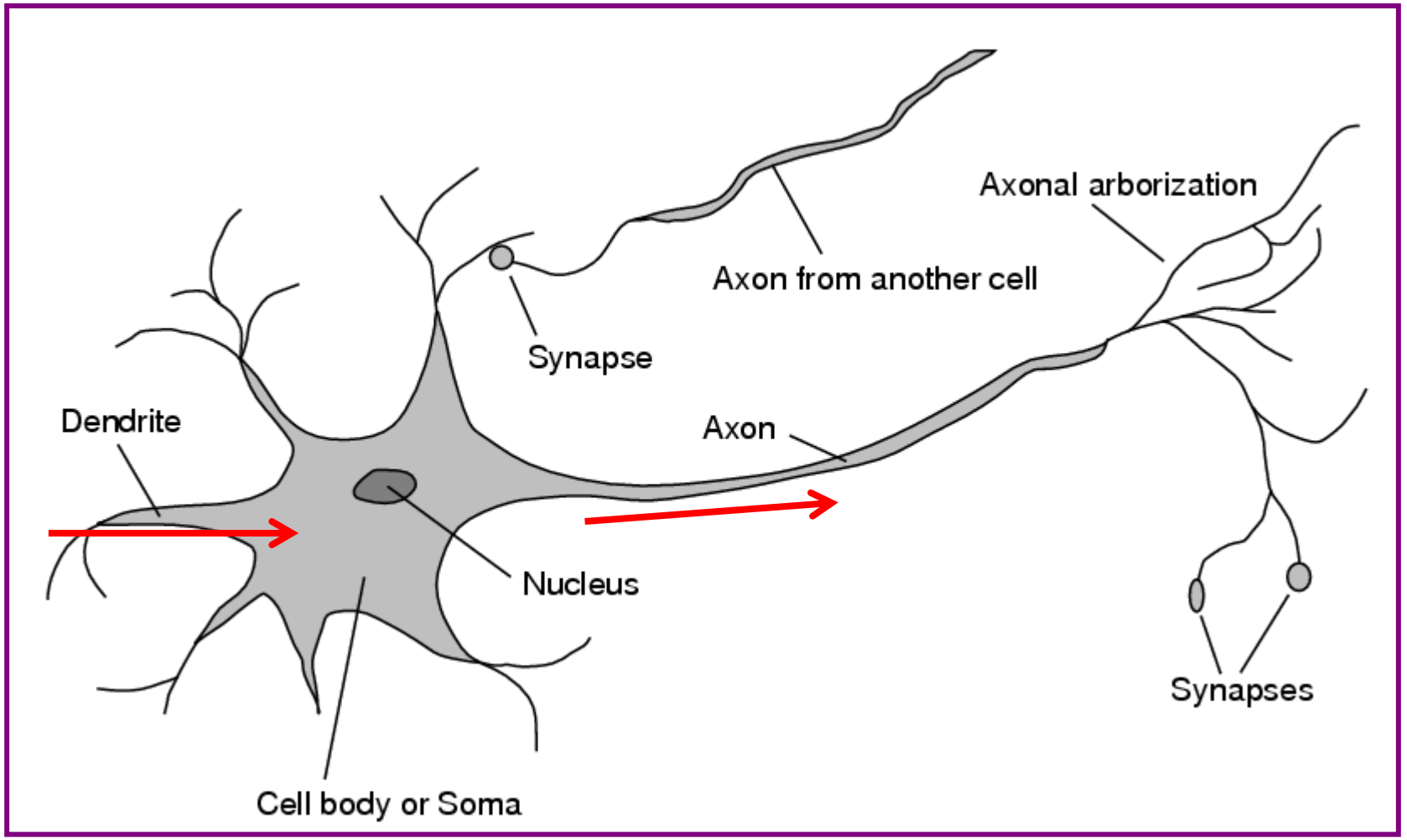


Features of the Brain

- In neural networks, a single percept (concept or pattern) is not stored in one neuron.
- Instead, it is represented by the combined activity of many neurons.
- Each neuron contributes partially to the overall representation.
- This is called distributed representation.



The Structure of Neurons



The Structure of Neurons

- **A neuron has**
- a cell body,
- a branching input structure (the dendrite) and
- a branching output structure (the axOn)

- Axons connect to dendrites via synapses.
- Electro-chemical signals are propagated from the dendritic input, through the cell body, and down the axon to other neurons

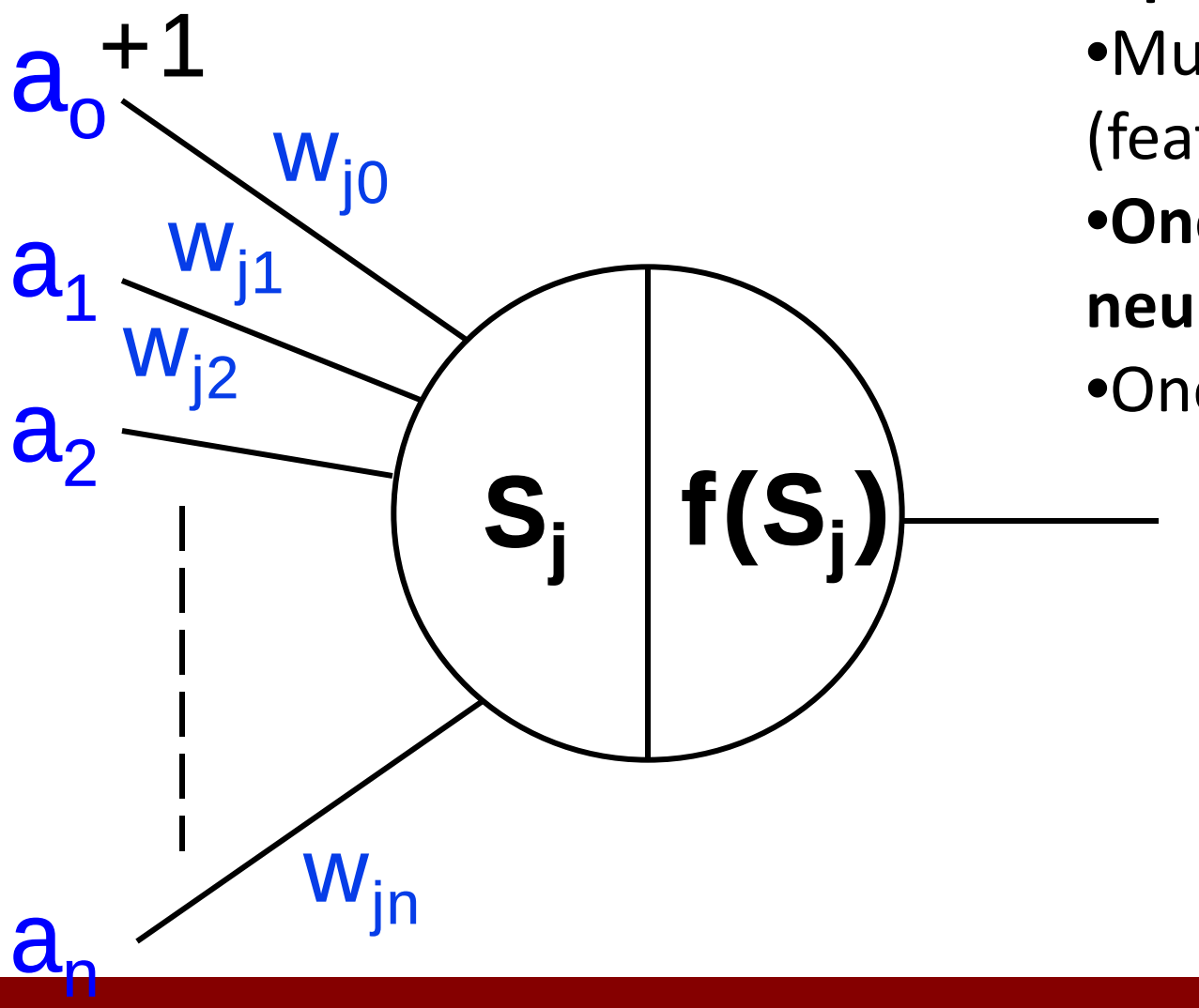
The Structure of Neurons

- A neuron only fires if its input signal exceeds a certain amount (the threshold) in a short time period.

- Synapses vary in strength (**connection weights**)
  - Good connections allowing a large signal
  - Slight connections allow only a weak signal

- Synaptic weight refers to the strength of the connection between two nodes, representing the degree of influence one neuron's activation has on another

The Artificial Neuron (Perceptron)



A **perceptron** consists of:

- Multiple **input nodes** (features)
- **One computational neuron (node)**
- One **output**

Perceptron

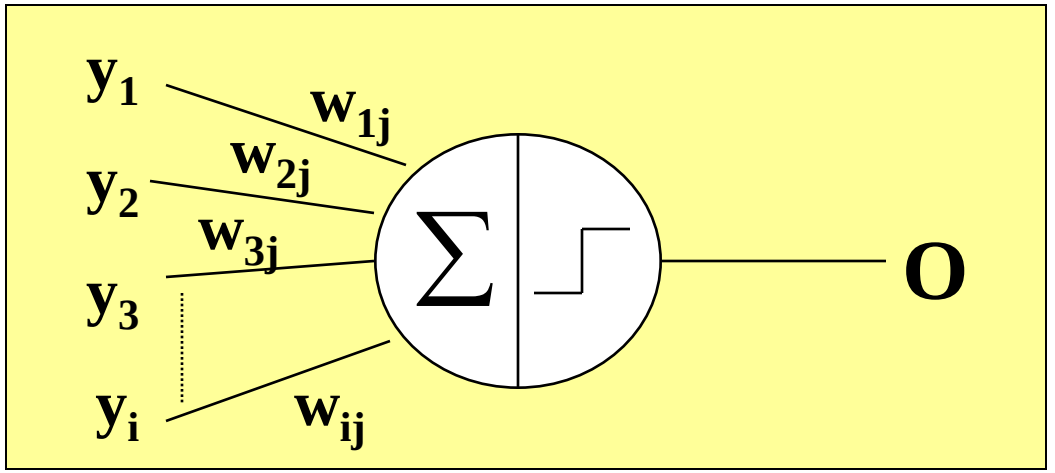
- The **perceptron** is an algorithm for supervised learning of binary classifiers

- Binary classifiers
  - functions that can decide whether an input, represented by a vector of numbers, belongs to some specific class or not.

- It is a type of linear classifier, i.e. a classification algorithm that makes its predictions based on a linear predictor function combining a set of weights with the feature vector.

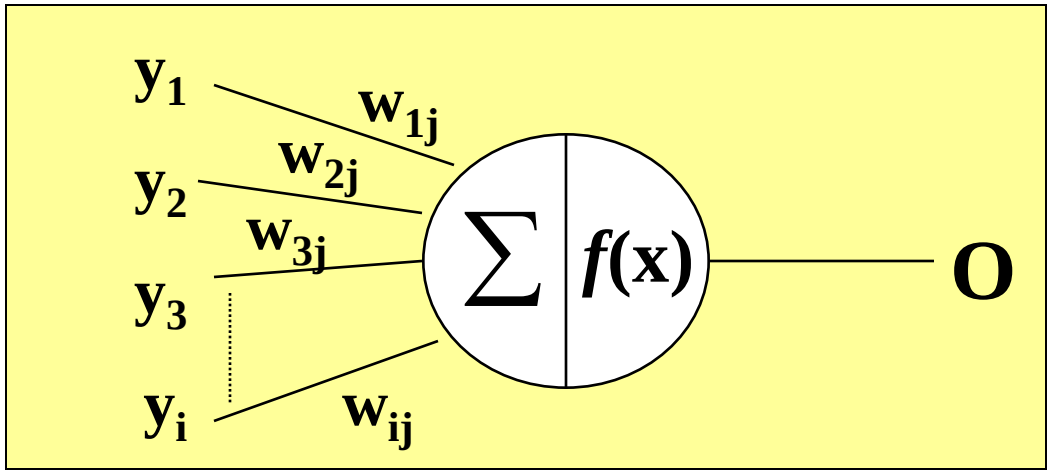
A Simple Model of a Neuron (Perceptron)

- **Input Neuron:**
  - Each neuron has a threshold value
  - Each neuron has weighted inputs from other neurons
  - The input signals form a weighted sum
  - If the activation level exceeds the threshold, the neuron “fires”



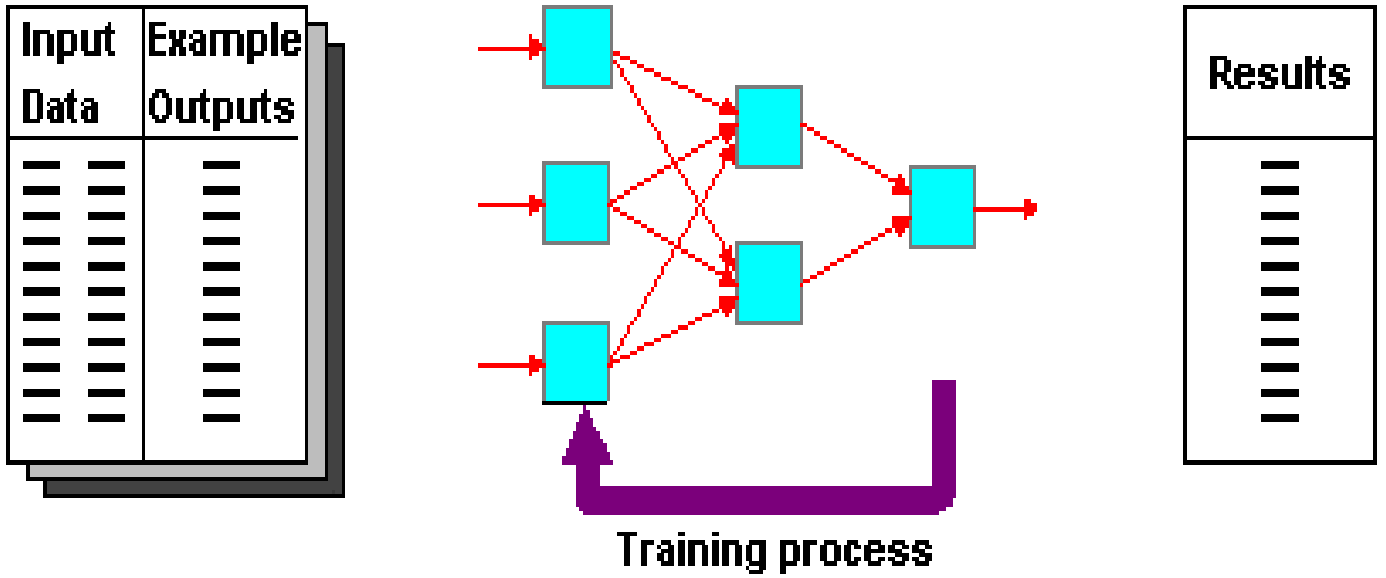
An Artificial Neuron

- **Hidden / Output Neuron:**
  - Each hidden or output neuron has weighted input connections from each of the units in the preceding layer.
  - The unit performs a weighted sum of its inputs, and subtracts its threshold value, to give its activation level.
  - Activation level is passed through a activation function (step, sigmoid etc) to determine output.

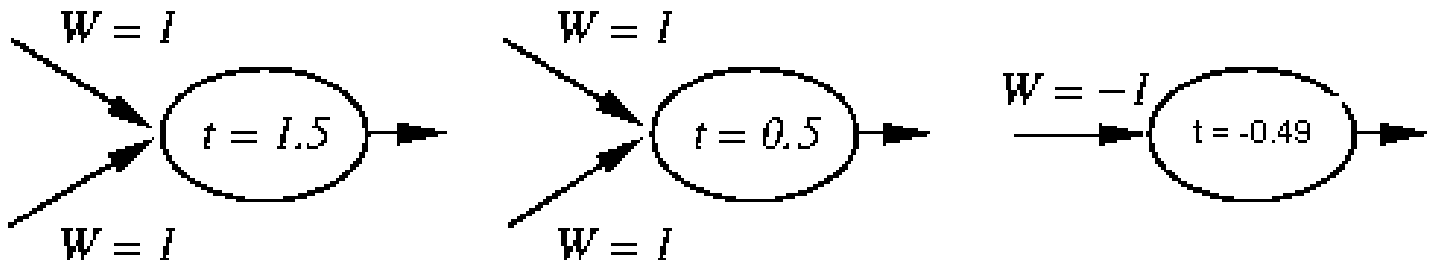


Supervised Learning

- Training and test data sets
- Training set; input & target



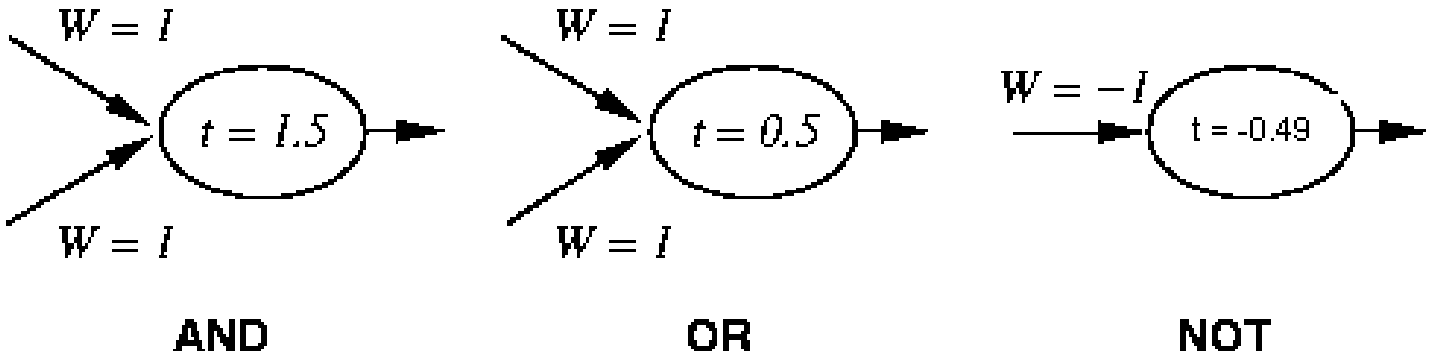
Perceptron Training



$$\text{Output} = \begin{cases} 1 \text{ if } \sum_{i=0} w_i x_i > t \\ 0 \text{ otherwise} \end{cases}$$

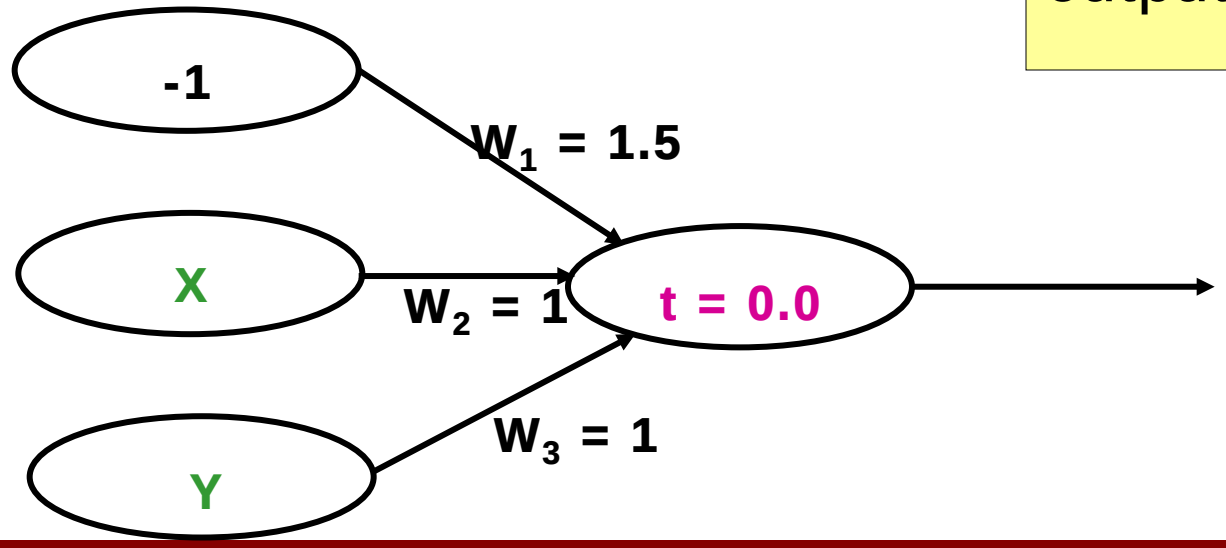
- Linear threshold is used.
- W - weight value
- t - threshold value

Simple Network



AND with a Biased input

$$\text{output} = \begin{cases} 1 \text{ if } \sum_{i=0} w_i x_i > t \\ 0 \text{ otherwise} \end{cases}$$



Learning algorithm

While epoch produces an error

Present network with next inputs from epoch

Error = T – O

If Error <> 0 then

$W_j = W_j + LR * I_j * Error$

End If

End While

Demo: https://www.cs.utexas.edu/~teammco/misc/perceptron/

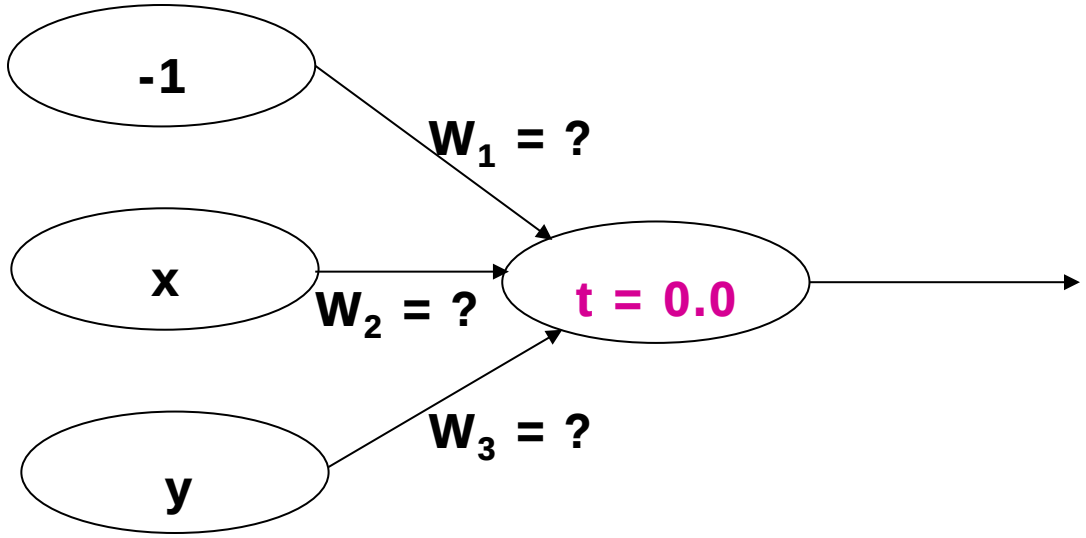
Learning algorithm

- **Epoch :** Presentation of the entire training set to the neural network.
  - In the case of the AND function an epoch consists of four sets of inputs being presented to the network (i.e. [0,0], [0,1], [1,0], [1,1])
- **Error**: The error value is the amount by which the value output by the network differs from the target value.
  - For example, if we required the network to output 0 and it output a 1, then Error = -1

Learning algorithm

- **Target Value, T** : When we are training a network we not only present it with the input but also with a value that we require the network to produce.
  - For example, if we present the network with [1,1] for the AND function the target value will be 1
- **Output , O** : The output value from the neuron
  - **Ij** : Inputs being presented to the neuron
  - **Wj** : Weight from input neuron (I_j) to the output neuron
  - **LR** : The learning rate. This dictates how quickly the network converges. It is set by a matter of experimentation. It is typically 0.1

Training Perceptrons

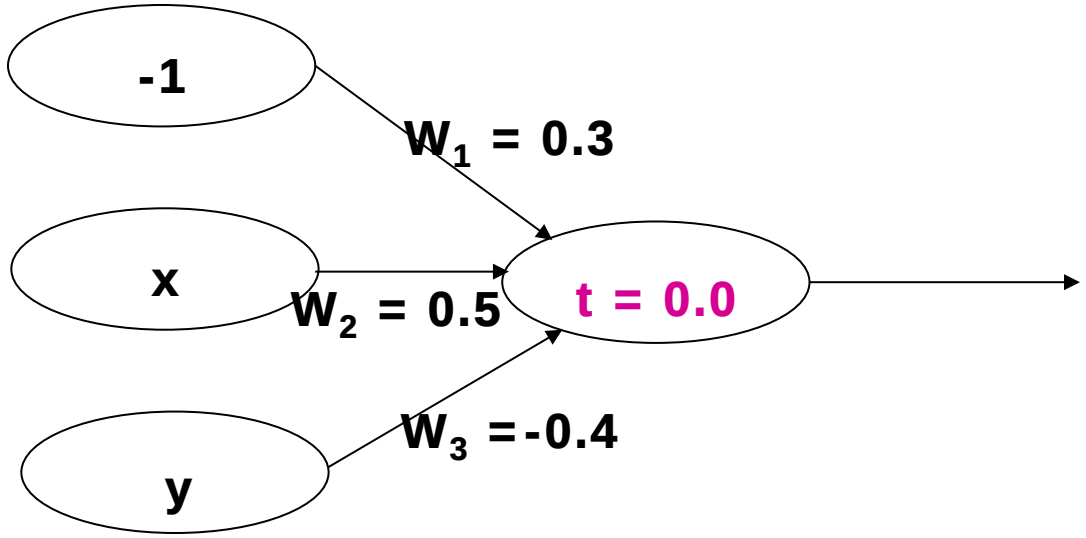


For AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

- What are the weight values?
- Initialize with random weight values

Training Perceptrons



For AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

I_1	I_2	I_3	Summation	Output
-1	0	0	(-1*0.3) + (0*0.5) + (0*-0.4) = -0.3	0
-1	0	1	(-1*0.3) + (0*0.5) + (1*-0.4) = -0.7	0
-1	1	0	(-1*0.3) + (1*0.5) + (0*-0.4) = 0.2	1
-1	1	1	(-1*0.3) + (1*0.5) + (1*-0.4) = -0.2	0

Learning in Neural Networks

- Learn values of weights from I/O pairs
- Start with random weights
- Load training example's input
- Observe computed output
- Modify weights to reduce difference
- Iterate over all training examples
- Terminate when weights stop changing OR when error is very small

Thank You